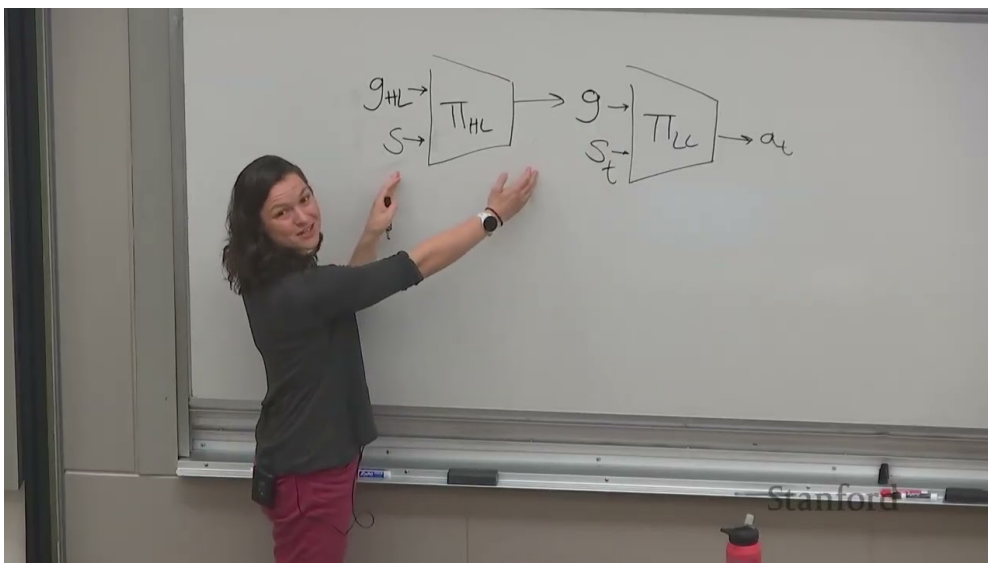


课程笔记

CS224R Lecture 15: 层次化模仿与强化学习

基于 Chelsea Finn 在 Stanford CS224R Lecture 15 的公开授课整理

2025 年 03 月 18 日



视频作者/频道: Stanford Online

发布日期: 2025 年 03 月 05 日

视频时长: 约 80 分钟

讲者: Chelsea Finn

课程主页: cs224r.stanford.edu

项目主页: <https://github.com/hqh1025/ai-course-notes>

目录

1 引言与内容概览	3
1.1 讲座定位与学习目标	3
1.2 时间线与素材地图	3
1.3 本章小结	4
2 层次化动机与任务拆解	4
2.1 长视野的典型陷阱	4
2.2 层次化与归纳偏置	5
2.3 工业场景中的分层模板	5
2.4 本章小结	5
3 Options 框架与 SMDP 实现	6
3.1 Semi-MDP 形式与建模准则	6
3.2 价值估计与 Bellman 方程	6
3.3 Option Pool 的维护与扩展	6
3.4 本章小结	7
4 Goal-Conditioned 分层策略	7
4.1 高层子目标与低层执行	7
4.2 训练挑战与反馈环	8
4.3 Hindsight relabeling 与数据重用	8
4.4 本章小结	8
5 层次化模仿与技能发现	9
5.1 从演示中提取技能	9
5.2 模仿数据治理	9
5.3 VQ-VAE 与离散潜空间	9
5.4 语言作为技能与命令	10
5.5 本章小结	10
6 语言与 LLM 驱动的高层调度	10
6.1 LLM 作为规划器的两个角色	10
6.2 Plan verification pipeline	11
6.3 治理与审计	11
6.4 本章小结	11
7 实践案例与工程治理	12
7.1 机器人长视野操作	12
7.2 评估与监控矩阵	12
7.3 Incident grade-book 与 lessons	13
7.4 本章小结	13

8 部署与复现保障	13
8.1 部署准备 checklist	13
8.2 可观测性与复现 tooling stack	14
8.3 本章小结	14
9 行动与落地建议	15
9.1 短期行动清单	15
9.2 长期研究议程	15
9.3 Checklist automation	15
9.4 本章小结	16
10 术语与指标速查	16
10.1 关键术语	16
10.2 指标矩阵与 dashboard	17
10.3 本章小结	17
11 Q&A 与实践提醒	17
11.1 现场问答要点	17
11.2 实践提醒	18
11.3 本章小结	18
12 总结与延伸	18
12.1 拓展阅读	19

1 引言与内容概览

1.1 讲座定位与学习目标

Lecture 15 由 Chelsea Finn 深度解析层次化模仿与强化学习的工程框架和治理路径。本章目的不仅是让我们理解为什么层次结构在长视野任务中不可或缺，更是去复现讲者在讲座中不断强调的“结构化探查 → 观察 → 治理”闭环。讲座横跨数据、架构、prompt/目标设计和实践案例，整个笔记按教学逻辑重组，方便日后复现。

构建二层次笔记的原则

将讲座拆解为：1) 动机、2) 框架、3) 子目标/模仿、4) 语言 + LLM 调用、5) 工程治理。每一块都需要用 ‘importantbox/knowledgebox/warningbox’ 强调洞察，用表格或图片整理对比，最后用小结归纳以便利复盘。

1.2 时间线与素材地图

时间区间	内容亮点与素材
00:00–00:12	讲者提出层次化原因、long horizon 与 hierarchical induction bias (slide-01 ~ 05)
00:12–00:30	Options/SMDP 形式化与 value estimation (slide-06 ~ 12)
00:30–00:45	Goal-conditioned 分层、HIRO/HAC 对比 (slide-13 ~ 20)
00:45–00:55	层次化模仿与技能发现 (slide-21 ~ 28)
00:55–01:12	LLM+ 语言规划、高级案例与部署建议 (slide-29 ~ 40)
01:12–01:20	实践 checklists + Q&A 收官 (slide-41 ~ 46)

表 1: Lecture 15 时间线与 slide 对应。

Hierarchy in Imitation and Reinforcement Learning

CS 224R

1

图 1: 讲座开场 slide: 层次结构为何值得关注 (slide-01)。

素材使用策略

优先引用 slides 截图；若 slide 内容不足，再用字幕/笔记扩展。每一幅图像必须铺设目录中的路径（如 ‘slides-images/slide-xx.jpg’），并在 caption 里列出 timecode 便于查回。

1.3 本章小结

本节通过定位、时间线和素材指引，构建了复盘本讲的地图。后续所有内容都依序围绕动机 → 模型 → 实践进行延展。

2 层次化动机与任务拆解

2.1 长视野的典型陷阱

单层策略面对稀疏奖励、组合爆炸与非平稳环境时常常陷入“微调-过拟合”的先验，Jason Punchline 是不放弃结构化视角。通过添加层次，我们重新定义动作空间的粒度：每个宏 action 负责若干步的子任务，降低信用分配压力。

长视野结构化分解的三步

1. 识别关键 check-point (example: 厨房任务的“取出锅”);
2. 提取每个 check-point 所需的 subgoal;
3. 设计子策略完成该 subgoal，并在完成后反馈全局 reward。

忽略分解的代价

- 在平坦策略中，长轨迹上任意一次失败都需要重新探索；
- Credit assignment 模型容易被噪音启动子任务，导致训练震荡；
- 没有分层的代理难以压缩上下文，面对部分可观测环境时崩溃。

2.2 层次化与归纳偏置

层次化在某种程度上也是归纳偏置——我们告诉模型“先完成一系列有意义的中间目标再接尾声”。正确地注入这样的偏置可以显著降低 sample complexity；但如果偏置不对（例如固定 skill 数量、缺少通用 stop condition），会形成 training bottleneck。

维度	传统 RL	层次化 RL
决策粒度	原子动作	子策略/宏动作
样本效率	低，需完整 rollout	高，子策略可复用
探索策略	纯探索	构造 subgoal 降低 search
信用分配	全局回报	子策略附加 reward

表 2: 传统 RL 与层次化 RL 任务拆解的对比。

2.3 工业场景中的分层模板

在实际产品（如厨房机器人、自动化测试平台）中，层次化模板通常包括：1) 感知/解析模块（抽取 subgoal），2) Strategy Planner（选 option/high-level plan），3) Execution Layer（skill + verifiers），4) Recovery + Logging（incident grade-book + prompt log）。每层都需把 input/output 明文记录到 ‘notes/ops/’，以便 on-call 团队快速对照。

常见工业模板

- Perception: state filtering + object detection;
- Planner: high-level policy 结合 options;
- Executor: skill library + environment interaction;
- Recovery: failure flag + fallback skill.

2.4 本章小结

动机部分强调“结构化”不是 add-on, 而是解决信用分配和 exploration 的核心; 后续的 Options、goal-conditioned 和模仿部分将围绕这两条主线展开。

3 Options 框架与 SMDP 实现

3.1 Semi-MDP 形式与建模准则

Options 框架将 MDP 扩展成 Semi-MDP，允许 policy 选择持续多个步的 macro-action。我们用 τ_ω 表示 option 执行的有效长度，期望 reward 以 $\mathbb{E}[\sum_{t=0}^{\tau_\omega} \gamma^t r_t]$ 建模，配合 discounted factor 保持收敛器。

Option 设计的三元组

- $\mathcal{I}_\omega$: 在哪些状态可触发 option;
- π_ω : 内部策略负责多步动作;
- β_ω : 终止函数控制 option 的 horizon。

3.2 价值估计与 Bellman 方程

Option 的 value 函数由 $Q(s, \omega) = \mathbb{E}[\sum_{t=0}^{\tau_\omega} \gamma^t r_t + \gamma^{\tau_\omega} V(s')]$ 表示。这促进我们在高层策略中进行 Q-learning (或 Actor-Critic) 而不用为每个原子动作做组合更新。

半马尔可夫 Bellman 展开

$$V(s) = \sum_{\omega} \pi_{\text{high}}(\omega|s) [\mathbb{E}_{\tau_\omega} [\sum_{t=0}^{\tau_\omega} \gamma^t r_t] + \mathbb{E}[\gamma^{\tau_\omega} V(s')]]$$

Option 过拟合问题

若 $\mathcal{I}_\omega$ 过窄或 π_ω 结构太强，高层策略会陷入“只有某几个 option 有效”的 local minima。建议采用 regularization (dropout style) 或动态扩展可用 option。

组件	作用
Initiation set	控制高层什么时候能选某 option
Intra-option policy	负责多步执行, 配合 skill replay
Termination function	决定是否继续保持 option
High-level policy	以粗时间粒度选 option

表 3: Options 框架的关键组件。

3.3 Option Pool 的维护与扩展

大型项目中需要维护一个 option pool: skill engineer 按照 coverage/usage matrix 更新 initiation set 与 termination 函数，并定期剔除低使用率或高 failure rate 的 option。可以把 usage log 写入 instrumentation dashboard，便于 governance review。

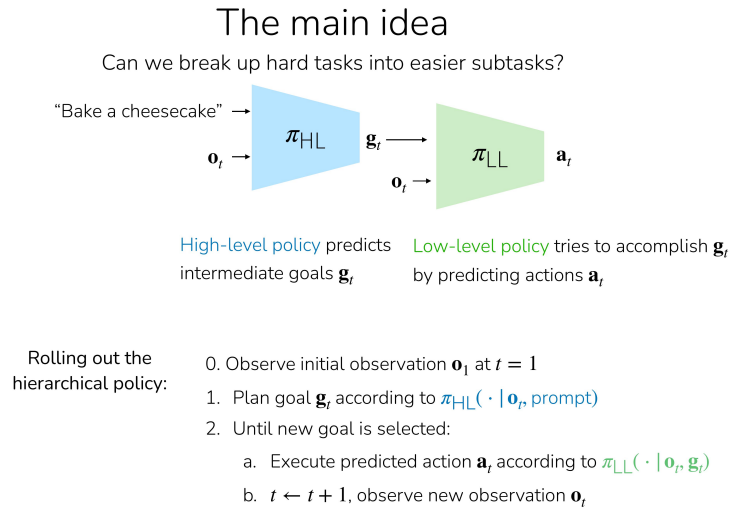


图 2: Options 训练流程与 usage tracking (slide-08)。

Option Pool 运维建议

- 用 coverage matrix 监督每个 option 被调用的次数；
- 每次 option 改动同步更新 prompt log 与 instrumentation；
- 对新 option 先在 sandbox 中测试，再拉入 production；

3.4 本章小结

Options/SMDP 提供了层次 RL 的数学规范：强调 action 的时间扩展和 value 函数的重写，并在实际项目中要求把 initiation/termination/logs 写入 instrumentation。

4 Goal-Conditioned 分层策略

4.1 高层子目标与低层执行

高层策略输出连续/离散子目标 g ，低层策略以 (s, g) 为 conditioning 进行动作采样。对于 continuous case，可把子目标定义为要素的高维 embedding；对于 discrete case，可直接当作技能 id。

方法	核心机制	典型处理
HIRO	Off-policy correction	高层每 c 步重设目标
HAC	Recursive hindsight relabeling	高层 reward 直接来自子目标
FuN	Predictive subgoal representations	用 predictive model 衡量 subgoal 可达性

表 4: 常见 goal-conditioned 分层方法对比。

子目标设计考量

- 可达性：子目标必须在低层 horizon 内可实现；
- 可辨识性：低层策略需要知道何时完成子目标；
- 可复用性：设计通用子目标，避免高层频繁创造新目标；
- 稳定性：高层子目标分布要有限，靠 hindsight relabel 控制 drift。

4.2 训练挑战与反馈环

层次 RL 的训练常伴随 non-stationarity：高层策略面对不断变化的低层行为，低层策略又依赖动态子目标。讲者建议使用双轨 replay buffer（分别存 high/low）并写 log 记录 high-level goal hash 与 associated reward。

梯度传播与回放冲突

- low-level replay 可能过拟合早期 high-level plan；
- high-level reward 需要依据 low-level completion 进行 hindsight relabel，否则梯度噪声过大；
- 使用 off-policy correction（如 HIRO）可校正 high-level update 的 bias。

4.3 Hindsight relabeling 与数据重用

Hindsight relabeling 把 high-level plan 中未达成的子目标，用 low-level 成功的状态“重写”为新的 pseudo-goal，从而让 replay buffer 提供更多正样本。实施时需通过 relabel scheduler 控制频率，避免过度 rewrite 破坏 original objective；所有 relabeled sample 的 metadata 应写入 prompt log 以便 audit。

Hindsight relabel 的实施步骤

1. 收集一条 episode 的高层 goal + 低层 trajectory；
2. 选择一个状态作为 relabel goal；
3. 重新计算 reward 并更新 high-level Q；
4. 将 relabeled sample 的 metadata（goal hash + timestamp）写入 instrumentation；

4.4 本章小结

Goal-conditioned 分层策略把高层像指挥官一样设子目标，低层视作执行者；训练难点在于频繁的环境/目标 drift，需要 hindsight+off-policy guardrail。

5 层次化模仿与技能发现

5.1 从演示中提取技能

模仿学习不再依赖 reward, 而是从 demo 中推断 skill segmentation。常见做法包括 sliding window clustering、change point detection、以及 skill-specific autoencoder。

技能发现 pipeline

1. 打标签: 把 demo 分割成 candidate skill;
2. 训练 skill policy: 把 segment 作为 policy context;
3. 训练 scheduler: 高层学习预测 skill sequence;
4. Deployment: 把 skill library 绑定 LLN/LLM 规划器。

5.2 模仿数据治理

治理模仿数据的 checklist

- Demo provenance: 记录来源视频/轨迹 id;
- Skill annotation: 标注子任务边界并导出 'labels.json';
- Diversity filter: 剔除过度重复的演示片段;
- Success flag: 每段演示标记是否完成, 供 low-layer reward 使用。

5.3 VQ-VAE 与离散潜空间

VQ-VAE 负责把 variable-length 轨迹压缩为 discrete codebook entry, 每个 entry 对应一个技能。高层策略只需预测 codebook index, 低层 decoder 即可“解码”出需要的动作序列。

Let's cover a few examples

1. Hierarchical [imitation learning](#)
 - a. with language subgoals
 - b. with image subgoals
2. Hierarchical [reinforcement learning](#)
 - a. with language subgoals
 - b. with state subgoals

图 3: Skill Discovery 的架构示意 (slide-25)。

潜变量 collapse

若 codebook size 太小, VQ-VAE 会反复复用同一技能, 导致高层策略缺乏多样性。常见解决方式是增加 commitment loss 或使用 EM-style clustering。

5.4 语言作为技能与命令

LLM 可以充当高层 policy 的语言 interface, 讲者提到 SayCan/Inner Monologue 通过 language → intent → skills pipeline 让低层执行器可解释性更高。

语言指令治理要点

- Prompt log 记录 language input + expected skill;
- 使用 parser 把 language 转换成 discrete skill id;
- 低层反馈 (例如 success flag) 需要回写到 prompt log, 形成 governance loop。

5.5 本章小结

模仿模块让我们可以从演示中自动发现技能库, 配合语言/LMM 接口则实现人机可解释的层次调度。

6 语言与 LLM 驱动的高层调度

6.1 LLM 作为规划器的两个角色

LLM 既可以输出子任务序列, 也可以做 feasibility filter。讲者强调要把 LLM 生成的 plan 打上 provenance (来源时间、prompt hash), 并对每个 step 进行 tool-check。

角色	实践要点
Plan generator	打上 prompt id、temperature、llm version
Verifier	依靠 tool chain 验证 plan 的可行性 (如 python sandbox)
Skill mapper	把 plan 转为 skill id 或 primitive action

表 5: LLM 在层次化流程中的多重身份。

LLM Prompt log

每条 prompt 对应 metadata: 'timestamp', 'llm-version', 'temperature', 'tool-call', 'expected skill id', 并将 hallucination/feasibility 反馈回 prompt log。

6.2 Plan verification pipeline

After fine-tuning high-level policy with DAgger
(the policy can self-correct)



Shi, Hu, Zhao, Sharma, Pertsch, Leo, Levine, Finn. Yell At Your Robot: Improving On-the-Fly from Language Corrections. RSS 2024

图 4: LLM 计划校验与 tool-check 流程 (slide-33)。

每条 LLM 输出先进入 verifier (python sandbox + knowledge graph), 再由 skill mapper 解析成可执行 plan。若发现 hallucination, 立刻通知 incident channel 并把 context 记录入 grade-book。

Plan verification 的 guardrail

- 每一条 plan 包含 plan_id、llm_version 与 prompt_hash;
- 执行前使用 tool-check 验证可行性;
- 发现 hallucination 时触发 rollback + 补 prompt 改动。

6.3 治理与审计

生成规划时要有治理 guardrail, 例如 multi-round verification、tool sandbox、human-in-the-loop review。若 LLM output 含 high-risk action (如 modify-files), 必须触发 incident drill。

LLM 计划调度的风险

自动化规划可能产生 hallucination, 尤其在多步骤任务中。建议在部署前设定:

- Multi-modal verification: 用 tool call verify each step;
- Stop tokens: 防止 LLM 无停止条件地追加 subtask;
- Human review: critical plan 前人工确认。

6.4 本章小结

语言 + LLM 让层次 RL 更具 explainability, 但也把 hallucination/incident 的治理压力前置到高层 prompt log 和 verification pipeline。

7 实践案例与工程治理

7.1 机器人长视野操作

Chelsea Finn 展示的厨房实例由多个 skill 和 safety guardrail 组成。我们将任务分成：1) 感知与状态估计，2) skill 执行（如拿起锅），3) verification + recovery。

Kitchen Robot 的监控点

- State check: 每步验证 gripper/contact 是否正常;
- Skill success flag: 低层 skill 完成后上报 ‘success’;
- Recovery template: 若 skill 失败，高层马上触发 safe fallback;
- Human override: 在 high-risk action (cutting/pouring) 前插入 human-in-the-loop。

7.2 评估与监控矩阵

引入 multi-metric scoreboard 以追踪 accuracy、self-consistency、hallucination rate、token budget 等。每次 emergent jump 都要同时检查多条曲线，避免因为 single metric 误判。

指标	描述	工具
Self-consistency	多路径生成的 agreement	CoT voting harness
Hallucination rate	事实性错误率	human eval + KG check
Token budget	资源消耗	monitoring hook
Instruction drift	Prompt log 版本偏移	versioned dashboard

表 6: 分层系统的评估矩阵。

忽略评估链路的后果

- 单看 accuracy 可能掩盖 hallucination;
- Prompt drift 会让 high-level plan 失效但低层仍然执行;
- 缺少 token budget 监控会造成 compute overrun。

7.3 Incident grade-book 与 lessons

字段	内容
Trigger	hallucination/jump/goal failure
Root cause	prompt drift / data shift / skill bug
Resolution	rollback prompt / retrain options / add guardrail
Next action	drill + doc update

表 7: incident grade-book 的模板。

Grade-book 的操作流程

- 24 小时内完成 trigger → analysis → resolution 的时间线；
- 把 prompt/data/attention snapshot 存入 ‘notes/incident/’；
- 把 lessons 更新到 ‘lessons.md’，在 weekly retro 分享。

7.4 本章小结

工程治理需要把 skill execution、LLM planning、监控矩阵串成闭环，并在每次 incident 中把 prompt log + evaluation snapshot 归档。

8 部署与复现保障

8.1 部署准备 checklist

部署前必须检视 prompt stability、token budget、fallback 方案与监控 hook。建议用 multi-column table 显示各项指标当前状态，并把每次检查的 artifact 写入 ‘deploy/logs/’。

检查项	验证方式	状态
Prompt stability	多 prompt + CoT + self-consistency	监听中
Token budget	latency + FLOPs shell	尺寸对齐
Fallback	offline fallback script + human review	预置
Monitoring hook	loss/latency/attention drift	hook enabled

表 8: 部署前 checklist。

准备 checklist 的使用

每次部署前用 standardized template 走一遍：prompt log → evaluation dashboard → fallback script。如果某项未通过必须记录在 incident grade-book 并延迟上线。

8.2 可观测性与复现 tooling stack

构建 tooling stack 时需包含 instrumentation、LLM plan 追踪、工具调用 trace (doc search、python executor 轮询) 和 attention/scoreboard snapshot。

Let's cover a few examples

- 1. Hierarchical imitation learning
 - a. with language subgoals
 - b. with image subgoals
- 2. Hierarchical reinforcement learning
 - a. with language subgoals
 - b. with state subgoals

37

图 5: Deployment + observability stack (slide-37)。

层级	工具/实践
Prompt	Prompt log + versioned dashboard
Inference	Monitoring hook + latency/-failure metrics
Tooling	API call trace + sandbox constraints
Observability	Attention/Config snapshot + drift alert

表 9: 部署部署可观测性 stack。

缺失 observability 的风险

- 没有 attention trace, 无法确认 emergent jump 是否真;
- 没有 tool call log, 低层 skill 无法追溯;
- 缺乏 drift alert, 将延迟 incident trigger。

8.3 本章小结

部署/复现部分是实践的护栏: 从 checklist 到 observability stack, 再到 incident grade-book, aim 是让 high-level insights 可落地。

9 行动与落地建议

9.1 短期行动清单

时间窗	可交付物
Week 1	完成 data review + skill segmentation; 设立 prompt log template
Week 2	Launch evaluation dashboard, 记录 multi-metric drift; 设定 incident grade-book
Week 3	触发一次 emergent drill: LLM plan + skill exec 与 governance review

表 10: 短期行动建议。

行动落地要点

- Prompt log + LLN plan 必须挂上 version/temperature/expected skill;
- 每次 emergent drill 都要有 review notes, 形成 lessons learned;
- Incident grade-book 记录 trigger、redirect action、后续 guardrail change。

9.2 长期研究议程

推荐以 quarterly thesis 方式梳理长期方向: 1)scale-law + hierarchical optimizer; 2)LLM-driven skill discovery; 3)attention interpretability for subgoal.

长期研究示例

- Project Alpha: 探索 hierarchical attention 模式;
- Project Beta: 把 scale law 曲线映射到 optimizer schedule;
- Project Gamma: 构建 emergent dashboard, 帮助 governance panel 决策。

9.3 Checklist automation

把短期 action checklist 自动化, 例如用 Github Actions 每周检查 prompt log 是否同步、用 dashboards 展示 multi-metric drift, 减少重复手动验证, 工程师可以把精力集中在 incident response 上。

自动化 checklist 提醒

- prompt 更新触发 `check_prompt_versions` workflow;
- drift alert 达到 threshold 触发 `incident-drill`;
- `summary.md` 每周生成 lessons 概览, 供 governance panel 审阅。

9.4 本章小结

行动建议部分把理论转化为 timeline: 短期 drill 快速闭环, 长期项目则把 emergent insights 打造成可验证的 research output。

10 术语与指标速查

10.1 关键术语

Hierarchical imitation with image subgoals

A few examples



Black*, Nakamoto*, Atreya, Walke, Finn, Kumar, Levine. SuSIE: Subgoal Synthesis via Image Editing. ICLR 2024.

39

图 6: 术语与指标速查 slide (slide-39)。

术语	释义
CoT	Chain-of-Thought, 向模型展示完整推理步骤
Self-consistency	多路径采样后取众数, 减少偶发错误
Governance loop	Prompt log + dashboard + incident grade-book
KV cache	Key/value 缓存, 用于 decoder-only 多轮对话加速

表 11: Lecture 15 重复出现的关键术语。

术语记忆建议

把关键术语写进 ‘glossary.md’ 并在 incident retro 时复盘, 帮助新加入的工程师快速上手。

10.2 指标矩阵与 dashboard

指标	作用
Self-consistency score	衡量 prompt/plan 稳定性
Hallucination rate	记录 fabrication 事件占比
Token budget	每次请求的 token/compute 限额
Attention entropy	判断模型是否跳到不同模式

表 12: 治理与调试时常用的指标。

指标滞后带来的风险

- 过度依赖 accuracy 可能掩盖 hallucination;
- Hallucination rate 如果 label delay, 会让 governance 反应滞后;
- 没有 attention entropy 监控, 很难确认 emergent jump 是否真实。

10.3 本章小结

把术语与指标写下来, 便于新成员快速理解讲座用语, 也可以在 incident review 中快速定位 key term + metric。

11 Q&A 与实践提醒

11.1 现场问答要点

在 Q&A 环节, Chelsea Finn 强调 “observe → instrument → govern” 三步, 特别是在 emergent drift 发生后要找出具体 prompt/data/attention artifact。

问题主题	核心回答
Emergent stability	多指标对齐后才能宣布 emergence 成熟
Prompt drift	insist on prompt log + rollback plan
Tooling stack	attention/plan/metric traces 必须 realtime + archival

表 13: Q&A 环节精华。

Q&A 记忆点

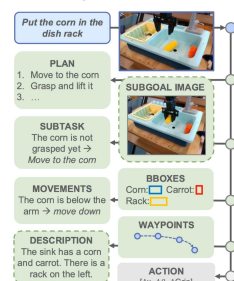
把 Q&A 中的 keyword 写进 ‘qa.md’, 每次 incident review 时对照, 确保没有遗漏高风险提醒。

11.2 实践提醒

Let's cover a few examples

1. Hierarchical **imitation learning**
 - a. with language subgoals
 - b. with image subgoals
2. Hierarchical **reinforcement learning**
 - a. with language subgoals
 - b. with state subgoals

Chain of thought version:



Visual from Chen, Belkhale, et al. *Training Strategies for Efficient Embodied Reasoning*, 2025.

Note: Fine-tuning large-scale hierarchical robot learning systems with RL is an open (and important!) research direction.

41

图 7: Q&A 与实践提醒 slide (slide-41)。

实践落地提醒

- 记录 prompt version + sampling seed;
- 每个 emergent drill 都要产出 lessons learned + summary note;
- 在 governance panel 会议上 review incident grade-book + prompt log。

11.3 本章小结

Q&A 让讲座的 abstract insights 成为 checklist; 只要把问答中的 artifact 持续写出来, 治理就始终有据可循。

12 总结与延伸

维度	核心复盘
动机	层次结构是解决信用分配与探索的关键
方法	Options/SMDP + goal-conditioned + imitation (VQ-VAE) 提供完整路径
语言治理	LLM prompt log + verification pipeline 保证高层计划可信
工程 评估	scoreboard + incident grade-book 让 emergent drift 可追踪

表 14: Lecture 15 三条主线回顾。

维度	风险与控制
非平稳性	hindsight relabel + off-policy correction
Hallucination	multi-metric governance + tool check
Script drift	versioned prompt log + incident reviews

表 15: 风险与治理补充视角。

12.1 拓展阅读

1. Sutton et al., “Between MDPs and Semi-MDPs: A Framework for Temporal Abstraction in RL,” 1999
2. Nachum et al., “Data-Efficient Hierarchical Reinforcement Learning (HIRO),” NeurIPS 2018
3. Pertsch et al., “Accelerating RL with Learned Skill Priors (SPiRL),” CoRL 2020
4. Ahn et al., “Do As I Can, Not As I Say: Grounding Language in Robotic Affordances (Say-Can),” 2022
5. Ajay et al., “Compositional Foundation Models for Hierarchical Planning,” NeurIPS 2023